Guía de laboratorio: APIs con Laravel

REST · GraphQL · WebSockets con Reverb · gRPC

Parte I — Tutorial paso a paso

Alcance y requisitos

Requisitos: PHP y Composer compatibles con la versión de Laravel instalada, Node.js y npm, SQLite o MySQL, Postman/curl, Python 3 (cliente gRPC) y terminales independientes. Se recomienda trabajar con Laravel 12/13 y verificar compatibilidad de Lighthouse y Reverb antes de instalar. El ejemplo es para desarrollo local, no para producción.

Arquitectura: REST y GraphQL llaman al mismo ProductoService; el servicio modifica Eloquent y dispara ProductoActualizado; Reverb transmite el evento; un servicio gRPC en Node.js consulta un endpoint interno protegido de Laravel. Así no se duplica la lógica.

1. Preparación del proyecto

```shell
composer create-project laravel/laravel inventario-api
cd inventario-api
```

```shell
php artisan install:api
```

```shell
php artisan make:model Producto -m
```

```shell
php artisan make:controller Api/ProductoController --api
php artisan make:class Services/ProductoService
```

```shell
php artisan make:event ProductoActualizado
```

Configurar DB_CONNECTION=sqlite en .env (y crear database/database.sqlite si es necesario), o configurar MySQL. Editar la migración de productos:

```php
Schema::create('productos', function (Blueprint $table) {
```

```php
    $table->id();
```

```php
    $table->string('nombre', 100);
```

```php
    $table->decimal('precio', 10, 2);
```

```php
    $table->unsignedInteger('stock')->default(0);
```

```php
    $table->timestamps();
```

```php
});
```

app/Models/Producto.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Producto extends Model
{
    protected $fillable = ['nombre', 'precio', 'stock'];
    protected $casts = ['precio' => 'decimal:2', 'stock' => 'integer'];
}
```

```shell
php artisan migrate
```

2. Servicio compartido: app/Services/ProductoService.php

Centraliza las validaciones, las operaciones y la emisión de eventos.

```php
<?php

namespace App\Services;

use App\Events\ProductoActualizado;
use App\Models\Producto;
use Illuminate\Support\Facades\Validator;
use Illuminate\Validation\ValidationException;

class ProductoService
{
    public function listar()
    {
        return Producto::orderBy('id')->get();
    }

    public function buscar(int $id): Producto
    {
        return Producto::findOrFail($id);
    }

    public function crear(array $datos): Producto
    {
        $datos = Validator::make($datos, [
            'nombre' => 'required|string|max:100',
            'precio' => 'required|numeric|min:0',
            'stock' => 'required|integer|min:0',
        ])->validate();

        $producto = Producto::create($datos);
        event(new ProductoActualizado($producto, 'creado'));
        return $producto;
    }

    public function actualizar(Producto $producto, array $datos): Producto
    {
        $datos = Validator::make($datos, [
            'nombre' => 'sometimes|required|string|max:100',
            'precio' => 'sometimes|required|numeric|min:0',
            'stock' => 'sometimes|required|integer|min:0',
```

```
        ])->validate();

        $producto->update($datos);
        event(new ProductoActualizado($producto, 'actualizado'));
        return $producto->refresh();
    }

    public function cambiarStock(Producto $producto, int $stock): Producto
    {
        return $this->actualizar($producto, ['stock' => $stock]);
    }

    public function eliminar(Producto $producto): void
    {
        $id = $producto->id;
        $producto->delete();
    }
}
```

Nota: En producción, para descontar stock de forma concurrente, usar transacciones y bloqueo de filas o una actualización atómica; el ejemplo actual asigna un valor absoluto de stock.

3. REST completo: controlador, rutas y respuestas HTTP

app/Http/Controllers/Api/ProductoController.php

PHP

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Producto;
use App\Services\ProductoService;
use Illuminate\Http\Request;

class ProductoController extends Controller
{
    public function __construct(private ProductoService $servicio) {}

    public function index()
    {
        return response()->json($this->servicio->listar(), 200);
    }

    public function store(Request $request)
```

```
    {
        $producto = $this->servicio->crear($request->all());
        return response()->json($producto, 201);
    }

    public function show(Producto $producto)
    {
        return response()->json($producto, 200);
    }

    public function update(Request $request, Producto $producto)
    {
        return response()->json(
            $this->servicio->actualizar($producto, $request->all()),
            200
        );
    }

    public function destroy(Producto $producto)
    {
        $this->servicio->eliminar($producto);
        return response()->noContent(); // 204
    }
}
```

routes/api.php

PHP

```php
<?php

use App\Http\Controllers\Api\ProductoController;
use Illuminate\Support\Facades\Route;

Route::apiResource('productos', ProductoController::class);
```

Laravel resuelve Producto por Route Model Binding; si no existe, devuelve 404. La validación fallida responde 422; crear devuelve 201; borrar devuelve 204 (sin cuerpo).

Pruebas REST

Shell

```shell
php artisan serve

# Ejecutar desde otra terminal:
curl -i http://127.0.0.1:8000/api/productos

curl -i -X POST http://127.0.0.1:8000/api/productos \
  -H 'Accept: application/json' -H 'Content-Type: application/json' \
```

```
  -d '{"nombre": "Teclado", "precio": 25000, "stock": 12}'

curl -i http://127.0.0.1:8000/api/productos/1

curl -i -X PATCH http://127.0.0.1:8000/api/productos/1 \
  -H 'Accept: application/json' -H 'Content-Type: application/json' \
  -d '{"stock": 11}'

curl -i -X DELETE http://127.0.0.1:8000/api/productos/1
```

4. GraphQL: consultas y mutaciones funcionales

Shell

```
composer require nuwave/lighthouse
php artisan vendor:publish --tag=lighthouse-schema
```

Nota: Si Composer reporta incompatibilidad, elegir una versión de Lighthouse que declare soporte para la versión de Laravel/PHP del equipo. El endpoint predeterminado suele ser POST /graphql.

graphql/schema.graphql

None

```
type Producto {
    id: ID!
    nombre: String!
    precio: Float!
    stock: Int!
}

type Query {
    productos: [Producto!]! @all(model: "App\\Models\\Producto")
    producto(id: ID! @eq): Producto @find(model: "App\\Models\\Producto")
}

type Mutation {
    crearProducto(nombre: String!, precio: Float!, stock: Int!): Producto!
@field(resolver: "App\\GraphQL\\Mutations\\ProductoMutator@crear")
    actualizarStock(id: ID!, stock: Int!): Producto! @field(resolver:
"App\\GraphQL\\Mutations\\ProductoMutator@stock")
}
```

En el archivo .graphql, mantener las barras invertidas escapadas dentro de los literales de cadena.

app/GraphQL/Mutations/ProductoMutator.php

```php
<?php

namespace App\GraphQL\Mutations;

use App\Services\ProductoService;

class ProductoMutator
{
    public function __construct(private ProductoService $servicio) {}

    public function crear($_, array $args)
    {
        return $this->servicio->crear($args);
    }

    public function stock($_, array $args)
    {
        $producto = $this->servicio->buscar((int) $args['id']);
        return $this->servicio->cambiarStock($producto, (int) $args['stock']);
    }
}
```

Crear manualmente el directorio app/GraphQL/Mutations si no existe. El resolver recibe el argumento raíz y el arreglo de argumentos; la lógica de validación sigue dentro de ProductoService.

Pruebas GraphQL en Postman (POST /graphql, Body JSON)

```json
{ "query": "{ productos { id nombre precio } }" }
{ "query": "mutation { crearProducto(nombre: \"Mouse\", precio: 18000, stock: 20) { id nombre stock } }" }
{ "query": "mutation { actualizarStock(id: 1, stock: 8) { id stock } }" }
```

Nota: Para evitar confusión con escapes al copiar: en Postman también puede seleccionar Body > GraphQL y pegar directamente la operación sin envolverla en JSON. Los errores GraphQL pueden devolverse con HTTP 200 y un campo errors: inspeccionar ambos.

5. WebSockets completos con Laravel Reverb

```shell
php artisan install:broadcasting --reverb
npm install
npm run dev
```

El instalador prepara Reverb y normalmente configura broadcasting, Echo y sus dependencias. Revisar .env y

resources/js/bootstrap.js según la versión instalada. El ejemplo usa canal público solo para datos de práctica.

app/Events/ProductoActualizado.php

```php
<?php

namespace App\Events;

use App\Models\Producto;
use Illuminate\Broadcasting\Channel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class ProductoActualizado implements ShouldBroadcast
{
    use Dispatchable, SerializesModels;

    public array $producto;
    public string $accion;

    public function __construct(Producto $producto, string $accion)
    {
        $this->producto = $producto->only('id', 'nombre', 'precio', 'stock');
        $this->accion = $accion;
    }

    public function broadcastOn(): array
    {
        return [new Channel('inventario')];
    }

    public function broadcastAs(): string
    {
        return 'producto.actualizado';
    }
}
```

resources/views/inventario.blade.php

```html
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Inventario en vivo</title>
```

```
    @vite(['resources/js/app.js'])
</head>
<body>
    <h1>Inventario en tiempo real</h1>
    <p>Abra esta página en dos pestañas y cambie el stock.</p>
    <table border="1" cellpadding="8">
        <thead>
            <tr>
                <th>ID</th>
                <th>Nombre</th>
                <th>Precio</th>
                <th>Stock</th>
                <th>Acción</th>
            </tr>
        </thead>
        <tbody id="productos"></tbody>
    </table>
    <pre id="logs"></pre>
    <script>
        async function cargar() {
            const res = await fetch('/api/productos');
            const productos = await res.json();
            document.getElementById('productos').innerHTML = productos.map(p =>
`
                <tr>
                    <td>${p.id}</td>
                    <td>${p.nombre}</td>
                    <td>${p.precio}</td>
                    <td>${p.stock}</td>
                    <td><button data-id="${p.id}" data-stock="${p.stock}">+1
stock</button></td>
                </tr>
            `).join('');
        }
        document.addEventListener('click', async e => {
            if (!e.target.matches('button[data-id]')) return;
            await fetch('/api/productos/' + e.target.dataset.id, {
                method: 'PATCH',
                headers: { 'Content-Type': 'application/json', 'Accept':
'application/json' },
                body: JSON.stringify({ stock: Number(e.target.dataset.stock) +
1 })
            });
            await cargar();
        });
        window.addEventListener('load', cargar);
        window.addEventListener('inventario-cambio', e => {
            document.getElementById('logs').textContent =
JSON.stringify(e.detail) + '\n' + document.getElementById('logs').textContent;
            cargar();
        });
    </script>
```

```
</body>
</html>
```

resources/js/app.js: agregar al final

```JavaScript
import './bootstrap';

window.Echo.channel('inventario')
    .listen('.producto.actualizado', (evento) => {
        console.log('Cambio recibido:', evento);
        window.dispatchEvent(new CustomEvent('inventario-cambio', { detail:
evento }));
    });
```

routes/web.php: agregar

```PHP
<?php

use Illuminate\Support\Facades\Route;

Route::view('/inventario', 'inventario');
```

Terminales para ejecutar

```Shell
# Terminal 1
php artisan serve

# Terminal 2
php artisan reverb:start

# Terminal 3 (ShouldBroadcast en cola)
php artisan queue:work

# Terminal 4
npm run dev
```

Abrir http://127.0.0.1:8000/inventario en dos pestañas. Pulsar +1 stock: la pestaña emisora refresca por HTTP y la otra debe recibir producto.actualizado por Reverb. Si no sucede, revisar consola del navegador, valores VITE_REVERB_*, REVERB_*, configuración de cola y procesos.

Nota: El botón +1 es demostrativo y no evita condiciones de carrera. No interpolar nombres sin sanitizar en innerHTML si los datos provienen de usuarios; para producción crear nodos DOM con textContent, autenticar operaciones y proteger canales.

6. gRPC funcional: contrato, servidor Node.js y cliente Python

Node.js actúa como servidor gRPC y consulta un endpoint interno de Laravel. Se utilizará una clave compartida de laboratorio para proteger ese endpoint.

Crear carpeta grpc e instalar dependencias

Shell

```
mkdir grpc
cd grpc
npm init -y
npm install @grpc/grpc-js @grpc/proto-loader
python -m pip install grpcio grpcio-tools
```

grpc/producto.proto

Protocol Buffer

```
syntax = "proto3";

package inventario;

service ProductoService {
    rpc ObtenerProducto (ProductoRequest) returns (ProductoResponse);
}

message ProductoRequest {
    int32 id = 1;
}

message ProductoResponse {
    int32 id = 1;
    string nombre = 2;
    double precio = 3;
    int32 stock = 4;
}
```

Laravel: endpoint interno protegido (routes/api.php)

PHP

```php
<?php

use App\Models\Producto;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::get('/interno/productos/{producto}', function (Request $request,
Producto $producto) {
    $clave = (string) env('GRPC_INTERNAL_KEY', '');
    abort_if($clave === '' || !hash_equals($clave, (string)
$request->header('X-Internal-Key')), 403);
    return response()->json($producto);
});
```

Agregar al .env de Laravel: GRPC_INTERNAL_KEY=clave-local-larga-de-prueba. Reiniciar php artisan serve tras modificar configuración; si existe caché, ejecutar php artisan config:clear. En producción usar config() y configuración fuera de rutas, autenticación robusta y red privada.

grpc/server.cjs

JavaScript

```javascript
const grpc = require('@grpc/grpc-js');
const loader = require('@grpc/proto-loader');
const path = require('path');

const def = loader.loadSync(path.join(__dirname, 'producto.proto'));
const api = grpc.loadPackageDefinition(def).inventario;

const server = new grpc.Server();
server.addService(api.ProductoService.service, {
    async ObtenerProducto(call, callback) {
        try {
            const url = 'http://127.0.0.1:8000/api/interno/productos/' +
call.request.id;
            const r = await fetch(url, {
                headers: {
                    'X-Internal-Key': process.env.GRPC_INTERNAL_KEY || ''
                }
            });
            if (r.status === 404) return callback({ code:
grpc.status.NOT_FOUND, message: 'Producto inexistente' });
            if (!r.ok) return callback({ code: grpc.status.INTERNAL, message:
'Fallo del backend Laravel: ' + r.status });
            const p = await r.json();
            callback(null, { id: p.id, nombre: p.nombre, precio:
Number(p.precio), stock: p.stock });
        } catch (err) {
            callback({ code: grpc.status.UNAVAILABLE, message: err.message });
        }
    }
```

```
});

server.bindAsync('127.0.0.1:50051', grpc.ServerCredentials.createInsecure(),
(err, port) => {
    if (err) throw err;
    console.log('Servidor gRPC en puerto', port);
});
```

grpc/client.cjs

JavaScript

```
const grpc = require('@grpc/grpc-js');
const loader = require('@grpc/proto-loader');
const path = require('path');

const def = loader.loadSync(path.join(__dirname, 'producto.proto'));
const api = grpc.loadPackageDefinition(def).inventario;

const client = new api.ProductoService('127.0.0.1:50051',
grpc.credentials.createInsecure());
client.ObtenerProducto({ id: 1 }, (err, producto) => {
    console.log(err || producto);
    client.close();
});
```

grpc/client.py (cliente Python opcional)

Python

```
import grpc
import producto_pb2
import producto_pb2_grpc

with grpc.insecure_channel('127.0.0.1:50051') as canal:
    stub = producto_pb2_grpc.ProductoServiceStub(canal)
    try:
        print(stub.ObtenerProducto(producto_pb2.ProductoRequest(id=1)))
    except grpc.RpcError as error:
        print(error.code(), error.details())
```

Generar stubs y ejecutar

Shell

```
# Dentro de grpc/
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=.
producto.proto

# Terminal Node (Linux/macOS):
GRPC_INTERNAL_KEY=clave-local-larga-de-prueba node server.cjs

# En PowerShell, alternativamente:
# $env:GRPC_INTERNAL_KEY='clave-local-larga-de-prueba'; node server.cjs

# En otras terminales:
node client.cjs
python client.py
```

Crear primero un producto con REST o GraphQL y comprobar que el ID solicitado existe. El cliente devuelve los mismos campos que REST, pero transportados como respuesta protobuf. gRPC usa HTTP/2 y en este ejemplo una conexión local sin TLS; nunca exponer el puerto inseguro públicamente.